ICT ACADEMY OF KERALA

SUMMER INTERNSHIP REPORT

SMART EMPLOYEE MANAGEMENT SYSTEM

Full-Stack Application Development Using React, TypeScript and Express

---

SUBMITTED BY

Mohamed Faris A

Sahrdaya College of Engineering and Technology

Academic Year 2026–27

EXECUTIVE SUMMARY

The Smart Employee Management System (Smart EMS) is a full-stack web application developed to make routine employee-record management more organized and accessible through a browser-based interface. The current implementation uses React and TypeScript for the frontend, Vite for development and build tooling, a Node.js and Express backend for API operations, and MongoDB Atlas for persistent data storage.

The application provides an employee directory with form-based creation and editing of employee details, along with search and status-filter controls that help users locate records more efficiently. The backend exposes employee-related API routes and uses Mongoose to communicate with MongoDB Atlas, while environment variables keep database configuration separate from the committed source code. Git and GitHub are used to track the application source and maintain the internship report in a separate repository.

The project demonstrates the integration of a modern frontend, a server-side API, and a cloud-hosted database within one practical application. Authentication, attendance and leave workflows, advanced reporting, a formal automated test suite, and a verified production deployment are not treated as completed features in this report because their implementation or successful verification has not been established.

1. INTRODUCTION

Employee details can become difficult to maintain when information is spread across spreadsheets or multiple separate records. Finding a particular employee, updating a job title, or checking a current employment status may take extra time when records are not presented in one consistent place. A centralized employee management interface can simplify these common activities by organizing employee information in a structured format and supporting basic record operations.

Smart EMS was developed as a practical full-stack application that separates the user interface from the backend services and database operations. The React interface sends requests to the Express API, and the backend uses Mongoose to work with MongoDB Atlas. The project therefore focuses on the core employee-directory workflow rather than attempting to represent a complete human-resources management platform.

2. PROJECT OBJECTIVES

- Develop a clear browser-based interface for viewing and maintaining employee records using React and TypeScript.
- Use Vite to support frontend development and create production build assets for the client application.
- Implement a Node.js and Express API that handles the employee-record operations supported by the application.
- Store employee information persistently in MongoDB Atlas and connect to the database through Mongoose.
- Provide forms for creating and editing records, alongside employee-directory search and status filtering.
- Use Git and GitHub for version control, repository hosting, and separation of the application code from the report.
- Keep database credentials and local configuration in an environment file instead of committing sensitive values to source control.

3. SCOPE AND DELIVERABLES

The current project scope covers the main employee-directory workflow. Users can view employee details such as name, department, job title, employment status, and joining date; open a form to add or edit employee information; search the directory; and filter visible records by status. The interface also includes summary cards and a confirmation prompt before deletion. Summary-card values are based on the records currently loaded by the frontend and should not be interpreted as a separately verified enterprise-wide analytics service.

The main deliverables are the React and TypeScript frontend, the Node.js and Express API, the employee model and route handlers, the employee form and directory interface, the MongoDB Atlas integration, the application repository, and this internship report. The current version should be described as an employee-record management application, not as a complete HR platform.

Features such as user authentication, role-based access control, attendance tracking, leave approval workflows, dedicated department administration, payroll analytics, CSV reporting, audit logs, and a formal automated integration-test suite remain outside the confirmed implementation scope and may be considered for future development.

4. TECHNOLOGY STACK

The project uses the following technology stack, with each component supporting a distinct part of the application:

Technology	Purpose in the Project
React	Builds the component-based user interface and supports interactive views and forms.
TypeScript	Adds static typing to frontend code and helps keep employee data structures consistent.
Vite	Provides the frontend development server and build tooling.
Node.js and Express	Run the backend server and expose API routes for employee operations.
MongoDB Atlas and Mongoose	Store employee data in the cloud database and provide the backend data-model and database-access layer.
Axios	Sends requests from the frontend to the backend API and receives employee records and operation responses.
Git and GitHub	Track source-code changes and host the application and report repositories.

5. SYSTEM ARCHITECTURE

The application follows a client-server architecture in which the frontend and backend run as separate parts of the system. The frontend presents employee information and collects input from the user, the Express API receives requests and performs the required operations, and the database layer stores and retrieves employee records.

FRONTEND	BACKEND	DATABASE
React + TypeScript Axios API requests	Node.js + Express Employee API routes	Mongoose + MongoDB Atlas Persistent employee records

The backend database connection is configured through the MONGODB_URI environment variable. Keeping this value in the local server environment file helps separate configuration from the public source code; the real .env file must not be committed to GitHub.

6. DEVELOPMENT METHODOLOGY

Development was carried out in stages, moving from project setup to interface development, API integration, database configuration, and troubleshooting. The work focused on connecting the main employee workflow end to end, while recording the boundaries of what was implemented and what still requires further testing.

Requirements and workflow planning. The planned workflow focused on viewing employee records, adding new records, editing existing details, deleting records with confirmation, and locating employees through search and status filters.

Frontend development. React and TypeScript were used to create the employee directory, the employee form, the dashboard summary cards, the search field, and the status-filter control. Vite was used to run the frontend during local development.

Backend development. Node.js and Express were used to create the server and employee API routes, allowing the frontend to communicate with the application logic through HTTP requests.

Database integration. Mongoose was used to connect the backend to MongoDB Atlas and work with the employee data model. The connection string is supplied through the MONGODB_URI environment variable rather than embedded directly in application code.

Frontend and API integration. Axios was used to send requests to the backend and display the returned employee records. Query parameters support the directory's search and status-filter behaviour.

Version control and debugging. Git and GitHub were used to track changes and maintain separate repositories for the application and the report. Troubleshooting included checking database configuration, resolving a frontend avatar-rendering issue, and investigating client-server communication.

7. IMPLEMENTED APPLICATION FEATURES

Employee directory. The main directory presents employee information in a structured view, helping users scan details such as employee identity, department, job title, employment status, and joining date without opening each record individually.

Employee form. The form includes fields for employee ID, first name, last name, email, phone number, department, job title, employment type, status, and joining date, and is used by the interface for creating and editing employee information.

Search and filtering. The application includes a search field and a status filter to help users narrow the displayed directory to records that match the selected criteria; the frontend passes the relevant parameters to the backend API.

Record deletion confirmation. Before a deletion is requested, the interface displays a confirmation prompt, helping reduce the possibility of removing an employee record unintentionally.

Database and configuration. Employee records are stored through the backend connection to MongoDB Atlas, while the database URI and other local configuration values are managed using environment variables.

8. REPOSITORY ORGANIZATION

The application source is organized into client and server areas to keep user-interface code separate from API and database logic. Key files identified during development include the following:

- client/src/App.tsx - main React interface and employee workflow views.
- client/src/api.ts - frontend API request functions.
- client/src/types.ts - TypeScript types used by the frontend.
- server/index.js - backend startup and database connection setup.
- server/models/Employee.js - employee data model.
- server/routes/employees.js - employee API route handlers.

9. RESULTS AND VERIFICATION

During local development, the backend was observed running at http://localhost:5000, and a successful connection to MongoDB Atlas was observed. The frontend was opened through the Vite development server, providing a working environment for checking the employee interface and its connection to the backend.

The add-employee workflow was confirmed to work locally. Other create, read, update, delete, search, and filtering paths still require additional end-to-end verification across the complete frontend and backend workflow. This report does not claim a formal automated test suite, a specific automated test-pass count, or a completed production deployment because those outcomes have not been verified.

The project's current outcome is a practical full-stack foundation for managing employee records. Further verification should confirm how each operation behaves with valid input, invalid input, missing fields, database errors, and network failures before the application is used beyond a development environment.

10. CHALLENGES AND LEARNING OUTCOMES

One challenge during development was ensuring that frontend fields and backend data access behaved consistently when employee details were incomplete or optional. An avatar-rendering issue occurred when the interface attempted to read the first character of a name that was not available; the display logic was adjusted to handle optional name values more safely.

The project also provided practical experience with MongoDB Atlas connection settings, environment variables, and the communication flow between a React frontend and an Express backend. Working through these issues helped demonstrate why the database connection string should remain outside committed source code and why API integration must be checked from both the client and server sides.

Through this work, the development process provided hands-on exposure to component-based UI development, TypeScript data types, REST-style API communication, database modelling, environment configuration, debugging, and version control. These are useful foundations for extending the project with more complete validation, testing, and deployment practices.

11. LIMITATIONS AND FUTURE ENHANCEMENTS

The current application is intentionally limited to the employee directory and its record-management workflow. Authentication and role-based permissions are not confirmed as implemented, so the application should not be presented as a secure multi-user HR system. Attendance and leave management, department administration, payroll, advanced reporting, audit trails, and automated test coverage also remain future enhancements.

Recommended next steps are to verify all CRUD and filter operations through end-to-end tests, add clear client-side and server-side validation, improve API error handling, and establish a repeatable automated test suite. After these checks, the project can be prepared for a production deployment with secure environment configuration, appropriate access controls, and a working public URL that has been tested.

12. CONCLUSION

The Smart Employee Management System demonstrates the essential structure of a full-stack web application by combining a React and TypeScript frontend, a Node.js and Express backend, and persistent data storage through MongoDB Atlas. Its employee directory and form-based record workflow provide a practical starting point for organizing employee information through a consistent interface.

The development process strengthened practical understanding of frontend state and forms, API integration, database configuration, debugging, environment-variable management, and version control. With additional testing, validation, access control, and deployment work, the current foundation can be extended into a more complete and reliable employee management solution.

APPENDIX A. PROJECT REPOSITORIES

Application repository: https://github.com/faris225043-hub/smart-employee.git

Internship report repository: https://github.com/faris225043-hub/smart-report.git

APPENDIX B. TECHNICAL REFERENCES

- React documentation: https://react.dev/
- TypeScript documentation: https://www.typescriptlang.org/docs/
- Vite guide: https://vite.dev/guide/
- Express documentation: https://expressjs.com/
- MongoDB Atlas documentation: https://www.mongodb.com/docs/atlas/
- Mongoose documentation: https://mongoosejs.com/docs/

APPENDIX C. LOCAL SETUP SUMMARY

To run the project locally, install Node.js, npm, and Git, and then obtain the project source from its GitHub repository. Configure the backend by creating server/.env from server/.env.example and supplying a valid MongoDB Atlas connection string in MONGODB_URI; keep this real environment file local and do not publish it to the repository.

Install the dependencies in the server and client folders, start the backend and frontend using the scripts defined in their respective package.json files, and then verify that the employee directory loads and that the frontend can communicate with the API. The exact commands should be checked against the scripts included in the current repository version.